

# SENG3320/6320 Software Verification and Validation

## Semester 1, 2019

### Assignment 2 – Automated Test Data Generation

25 marks (25% of the whole course assessment)  
Due 11:59pm, Friday, 7 June 2019

#### 1. Fuzz Testing (11%)

Fuzz Testing is a random testing technique, which is commonly used to discover software crashes through a large amount of random data. It is a cost-effective alternative to more systematic testing techniques.

In this assignment, you are to apply fuzz testing to test a KWIC program. The KWIC (Key Word In Context) problem is defined as follows:

-----  
“The KWIC [Key Word in Context] index system accepts an ordered set of lines; each line is an ordered set of words, and each word is an ordered set of characters. Any line may be “circularly shifted” by repeatedly removing the first word and appending it at the end of the line. The KWIC index system outputs a list of all circular shifts of all lines in alphabetical order.”

The KWIC system is a real system and is widely used in keyword in context indexes for libraries.

#### **Example**

Consider the following three book titles (lines):

- Pattern-Oriented Software Architecture
- Software Architecture
- Introducing Design Patterns

The KWIC system produces the following output:

- Architecture Software
- Architecture Pattern-Oriented Software
- Design Patterns Introducing
- Introducing Design Patterns
- Patterns Introducing Design
- Pattern-Oriented Software Architecture
- Software Architecture
- Software Architecture Pattern-Oriented

Users can now quickly search for book titles that contain phrases such as “Software Architecture” or “Design Pattern”.

-----  
For this assignment, you are given a Java implementation of the KWIC problem (KWIC.class), which can be found in Blackboard. The usage of this program is:

*java KWIC input.txt*

where input.txt is a plain text file containing the input book titles (each line is a title).

You are required to apply fuzz testing to generate test input to crash the given KWIC program. More specifically, you are required to:

- Perform manual black-box testing (manual test case design based on the problem specification)
- Develop a fuzz testing tool, which can feed a large amount of random data (random book titles such as “a8h h19%p”) to the KWIC program in an attempt to make it crash.
- Perform the fuzz testing, record the number of unique crashes (crashes with different exception messages at different program locations) and the input that crashes the program.
- Write a test report, which describes the test tool design, test environment, example of test cases, and summary of test results (especially the number of unique crashes detected).

The marks will be distributed as follows:

|                                              |         |
|----------------------------------------------|---------|
| Manual black-box testing of the KWIC program | 2 marks |
| Fuzz tool development                        | 3 marks |
| Fuzzing testing of the KWIC program          | 3 marks |
| Test report                                  | 3 marks |

## 2. Search-based Software Testing (7%)

As described in the lecture, search-based software testing formulates test case generation as a search problem, and applies various metaheuristic algorithms (such as Hill climbing and genetic programming) to automatically generate test data. In this assignment, you are to experiment with search-based test data generation using EvoSuite, a search-based software testing tool that utilizes genetic algorithm. More specifically, you will:

- Download and install the EvoSuite tool from <http://www.evosuite.org/downloads/>
- Choose a GitHub project at: <http://www.evosuite.org/experimental-data/github-subjects/> (you can choose any one project from the list).
- Apply EvoSuite to generate test data for the GitHub project you selected.
- Compute the coverage of the test data, including class, method, statement, and branch decision coverage. Class/method coverage means the percentage of Java classes/methods covered by the test data.
- Write a test report for this experiment. The test report should describe your experimental design, steps, and results.

The marks will be distributed as follows:

|                                                                         |         |
|-------------------------------------------------------------------------|---------|
| Successful application of EvoSuite to the testing of the GitHub project | 3 marks |
| Coverage analysis                                                       | 2 marks |
| Test report                                                             | 2 marks |

## 3. Symbolic Execution (7%)

As described in the lecture, symbolic execution is a means of analyzing a program to determine what inputs cause each part of a program to execute. It ‘executes’ programs with symbols rather than concrete input. In this assignment, you are to experiment with symbolic execution based test data generation using KLEE, a symbolic execution tool for C programs. More specifically, you will:

- Download and install the KLEE tool from <http://klee.github.io/>
- Apply KLEE to generate test data for the Median program as shown below.

- Compute the control-flow coverage of the test data, including statement, branch decision coverage, condition coverage, and condition/decision coverage.
- For all the input variables, identify the P-Uses (i.e., predicate-use) and C-Uses (i.e., computation-use), and compute the coverage of P-Uses and C-uses achieved by the test data (data-flow coverage).
- Write a test report for this experiment. The test report should describe your experimental design, steps, and results.

```
// The Median program
int median(int x, int y, int z){
    int median = 0;
    if(x >= y && x <= z){ // y<=x<=z
        median = x;
    } else if(x >= z && x <= y){ // z<=x<=y
        median = x;
    } else if(y >= x && y < z){ // x<=y<=z
        z = y;           // a bug here
    } else if(y >= z && y <= x){ // z<=y<=x
        median = y;
    } else { // x<=z<=y or y<=z<=x
        median = z;
    }
    return median;
}
```

The marks will be distributed as follows:

|                                                                     |         |
|---------------------------------------------------------------------|---------|
| Successful application of KLEE to the testing of the Median program | 2 marks |
| Control flow coverage analysis                                      | 2 marks |
| Data flow coverage analysis                                         | 2 marks |
| Test report                                                         | 1 mark  |

This assignment should be submitted via Blackboard. The submission should be a single zip file, which contains all source code, program under test, test data, test report, and supporting materials. Copying, plagiarism and other malpractices are not allowed. Write in your own words. Please remember to attach an Assignment Cover Sheet to each submission.

Start early and work efficiently and effectively with your group members!

- End -